

Data Structure

COURSE CODE: CS-201

TOPIC: CIRCULAR LINK LIST

INSTRUCTOR: HABIBA ARSHAD



Goals for today

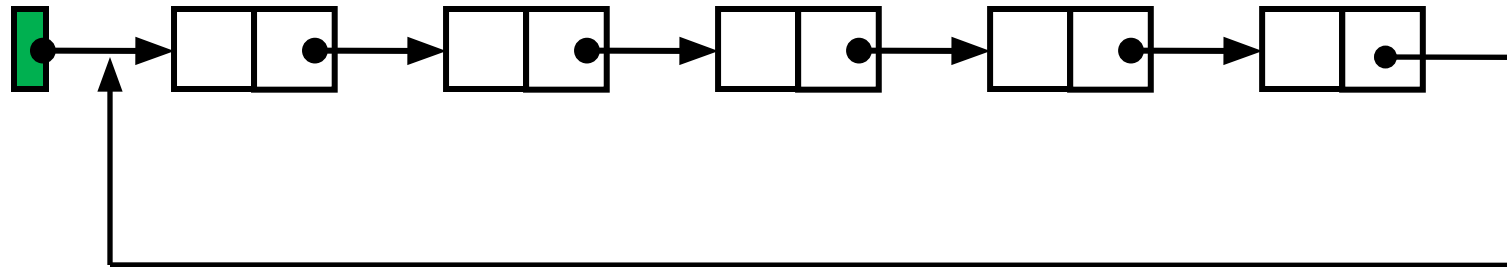
To study:

- Circular Linked list

CLO Covered

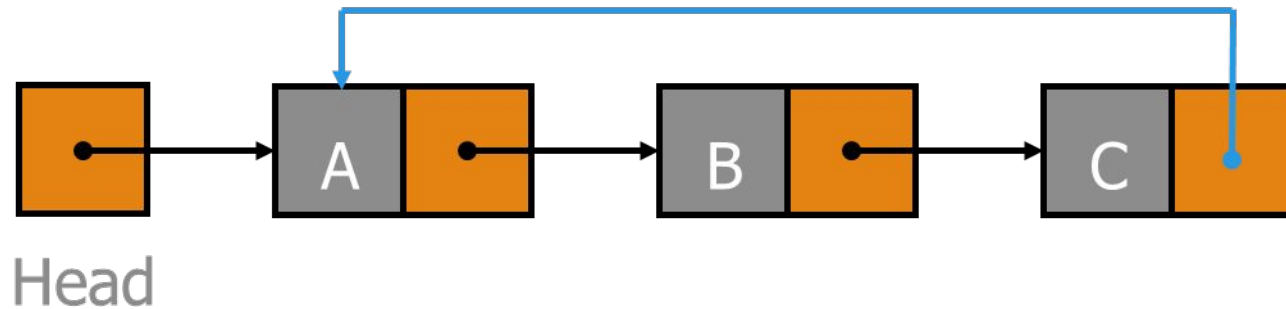
CLO2: Apply and design various data structures methods for specified applications.

Other variation of Linked List



Circular Linked Lists

- A linked list whose last node points back to the first node instead of containing a NULL pointer called a **circular list**



- How do we know when we have finished traversing the list? (Tip: check if the pointer of the current node is equal to the head.)

Header Linked Lists

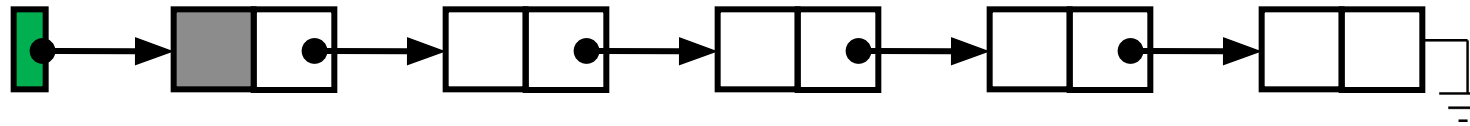
A **header linked** list is a linked list which always contains a **special node** called **header node** at the beginning of the list.

Types of header list:

- **Grounded header list**
- **Circular header list**

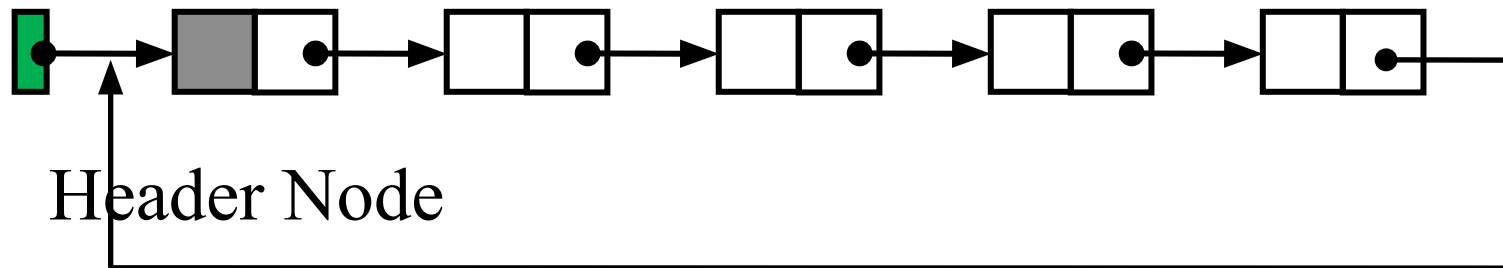
Types Header Linked Lists

Grounded Header List: A header list where the last node contains the NULL pointer.



Header Node

Circular Header List: A header list where the **last node** points back to the **header node**.



Header Node

Application/ uses

- Circular lists are used in applications where the entire list is accessed one-by-one in a loop.
- Example: Operating systems may use it to switch between various running applications in a circular loop.
- Multiplayer games uses circular list to swap between players in a loop.

Traversing a Circular Header List

Let **Head** be a circular header list in memory. Write an algorithm to **print Data** in each node in the list.

Algorithm for Traversing a Circular Header List

```
i = head;  
do {  
    print i->data;  
    i = i->link;  
} while (i != head);
```

Locating an ITEM

Let **Head** be a circular header list in memory.

Write an algorithm to find the location LOC of the first node in Head which contains ITEM or return LOC = NULL when the item is not present.

Algorithm for Locating an ITEM

```
i = head;
do
{
    if (i->data == Item)
    {
        loc = i;
        print i->data;
        break;
    }
    i = i->link;
} while (i != head);
```

Your turn

Create an order management system for a restaurant using a doubly linked list. Each node represents a customer order with the attributes/data as (orderId, customername, orderDetails, next, and prev).

Write an algorithm to add a new order at the end of the list. If there are no orders, the new order should be the first node. Also write procedure to create structure for node.

```
struct Node {  int orderID;  
string customerName;  
string orderDetails;  
Node* next;  
Node* prev; };
```

```
Node *head = nullptr;
```

```
Node *tail = nullptr;
```

Create first node

```
if(head==NULL)
```

```
Node* newNode = new struct Node;
```

```
newNode->orderID = orderID;
```

```
newNode->customerName = customerName;
```

```
newNode->orderDetails = orderDetails;
```

```
newNode->next = nullptr;
```

```
newNode->prev = nullptr;
```

```
head = newNode;
```

```
tail = newNode;
```

Create node at the end

```
else
```

```
Node *newNode, *p
```

```
newNode = new Node;
```

```
newNode->orderID = orderID;
```

```
newNode->customerName = customerName;
```

```
newNode->orderDetails = orderDetails;
```

```
if(head == nullptr){
```

```
    create first node
```

```
}
```

```
p=head
```

```
While(p->next !=Null){
```

```
    p=p->next
```

```
}
```

```
P->next = newNode
```

```
newNode->next=NULL
```

```
newNode->prev=p
```

```
Tail = newNode
```

Questions?